UD4. Manipulación de datos e xestión de transaccións

Bloque 1. Manipulación de datos (DML)

Neste bloque estudaranse as operacións que permiten inserir, modificar e eliminar rexistros nunha base de datos relacional. Tamén se analizará como as claves primarias, as claves foráneas e as regras de integridade referencial condicionan estas operacións.

> **Entorno de traballo:** Os exemplos utilizan MariaDB como sistema xestor de bases de datos e MySQL Workbench como ferramenta cliente para executar as sentenzas SQL.

1. Introdución á manipulación de datos

1.1. Que é a manipulación de datos?

Unha base de datos almacena información organizada en táboas. Unha vez creadas as táboas e definidas as súas relacións, é necesario realizar operacións para incorporar novos rexistros, modificar información existente ou eliminar datos que xa non sexan necesarios.

Estas operacións realízanse mediante sentenzas SQL. As instrucións utilizadas para inserir, actualizar e eliminar información pertencen ao grupo denominado **DML** (*Data Manipulation Language*), é dicir, linguaxe de manipulación de datos.

1.2. Diferenzas entre DDL, DML e DCL

SQL inclúe diferentes grupos de sentenzas segundo a súa finalidade. É importante distinguir entre modificar a estrutura dunha base de datos, modificar os datos almacenados e administrar os permisos dos usuarios.

Grupo	Finalidade	Sentenzas habituais
DDL Data Definition Language	Crear, modificar ou eliminar estruturas da base de datos, como bases de datos, táboas, columnas, índices e restricións.	`CREATE`, `ALTER`, `DROP`, `TRUNCATE`
DML Data Manipulation Language	Inserir, modificar e eliminar rexistros almacenados nas táboas.	`INSERT`, `UPDATE`, `DELETE`
DCL Data Control Language	Administrar os permisos e privilexios dos usuarios sobre os obxectos da base de datos.	`GRANT`, `REVOKE`

As consultas realizadas mediante `SELECT` adoitan clasificarse nun grupo específico denominado DQL (*Data Query Language*), aínda que frecuentemente se estudan xunto coas operacións de manipulación de datos.

Exemplo: diferenzas entre os grupos de sentenzas

```
-- DDL: crea unha táboa
CREATE TABLE categorias (
    id INT PRIMARY KEY,
    nome VARCHAR(50) NOT NULL
);

-- DML: insire un rexistro
INSERT INTO categorias (id, nome)
VALUES (1, 'Informática');

-- DML: modifica un rexistro
UPDATE categorias
SET nome = 'Sistemas informáticos'
WHERE id = 1;

-- DML: elimina un rexistro
DELETE FROM categorias
WHERE id = 1;

-- DCL: concede un permiso
GRANT SELECT ON academia.categorias
TO 'consulta'@'localhost';
```

1.3. Ferramentas para modificar datos

As operacións DML poden executarse utilizando diferentes ferramentas:

- **Consola de MariaDB:** permite conectarse directamente ao servidor e executar sentenzas SQL mediante unha terminal.
- **MySQL Workbench:** ofrece un editor SQL desde o que se poden escribir e executar sentenzas, consultar resultados e revisar erros.
- **Aplicacións:** os programas desenvolvidos en linguaxes como PHP, Java ou Python poden conectarse ao servidor e executar operacións DML mediante os seus controladores.

A ferramenta utilizada non modifica o significado da sentenza SQL. Unha mesma instrución `INSERT` executarase de maneira equivalente desde a consola ou desde Workbench, sempre que se utilice a mesma conexión e o usuario dispoña dos permisos necesarios.

1.4. Seleccionar a base de datos de traballo

Antes de executar as sentenzas, é necesario seleccionar a base de datos sobre a que se vai traballar. En MariaDB pódese facer mediante a instrución `USE`.

```
-- Consultar as bases de datos existentes
SHOW DATABASES;

-- Seleccionar a base de datos
USE academia;

-- Comprobar a base de datos seleccionada
SELECT DATABASE();
```

En MySQL Workbench tamén se pode seleccionar o esquema no panel de esquemas. É importante comprobar sempre que se está traballando sobre a base de datos correcta.

2. Base de datos de exemplo

Para os exemplos deste bloque utilizarase unha pequena base de datos de xestión académica. Estará formada polas seguintes táboas:

- `cursos`: almacena os cursos dispoñibles.
- `alumnos`: almacena os datos do alumnado.
- `asignaturas`: almacena as materias impartidas.
- `matriculas`: relaciona o alumnado coas asignaturas.

As relacións entre estas táboas son:

- Un curso pode ter moitos alumnos.
- Un alumno pertence a un curso.
- Un alumno pode matricularse en varias asignaturas.
- Unha asignatura pode ter varios alumnos matriculados.

Creación da base de datos e das táboas

```
CREATE DATABASE IF NOT EXISTS academia;

USE academia;

CREATE TABLE cursos (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nome VARCHAR(20) NOT NULL UNIQUE
) ENGINE=InnoDB;

CREATE TABLE alumnos (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nome VARCHAR(80) NOT NULL,
    email VARCHAR(120) UNIQUE,
    data_alta DATE NOT NULL,
    activo BOOLEAN NOT NULL DEFAULT TRUE,
    curso_id INT,

    CONSTRAINT fk_alumnos_cursos
        FOREIGN KEY (curso_id)
        REFERENCES cursos(id)
) ENGINE=InnoDB;

CREATE TABLE asignaturas (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nome VARCHAR(80) NOT NULL UNIQUE
) ENGINE=InnoDB;

CREATE TABLE matriculas (
    alumno_id INT NOT NULL,
    asignatura_id INT NOT NULL,
    data DATE NOT NULL,
    nota DECIMAL(4,2),

    PRIMARY KEY (alumno_id, asignatura_id),

    CONSTRAINT fk_matriculas_alumnos
        FOREIGN KEY (alumno_id)
        REFERENCES alumnos(id),

    CONSTRAINT fk_matriculas_asignaturas
        FOREIGN KEY (asignatura_id)
        REFERENCES asignaturas(id),

    CONSTRAINT chk_nota
```

```
        CHECK (nota IS NULL OR (nota >= 0 AND nota <= 10))
) ENGINE=InnoDB;
```

> **Importante:** Utilízase o motor de almacenamento `InnoDB`, que admite claves foráneas e transaccións. As restricións definidas nas táboas son comprobadas polo servidor cando se executan as operacións DML.

3. Inserción de rexistros: INSERT

3.1. Sintaxe básica

A sentenza `INSERT INTO` permite engadir novos rexistros a unha táboa. Na súa forma básica, indícase a táboa, as columnas que recibirán os datos e os valores correspondentes.

```
INSERT INTO nome_taboa (columna1, columna2, columna3)
VALUES (valor1, valor2, valor3);
```

Os valores deben corresponderse coas columnas indicadas e respectar os seus tipos de datos e as restricións definidas na táboa.

3.2. Inserir un rexistro

Exemplo

```
INSERT INTO cursos (nome)
VALUES ('DAW1');

INSERT INTO cursos (nome)
VALUES ('DAM1');

INSERT INTO asignaturas (nome)
VALUES ('Bases de datos');
```

Como o campo `id` está definido como `AUTO_INCREMENT`, MariaDB xera automaticamente o seu valor. Non é necesario indicalo na sentenza.

3.3. Consultar os datos inseridos

Despois de inserir datos, pódese comprobar o resultado mediante unha consulta `SELECT`.

```
SELECT * FROM cursos;

SELECT * FROM asignaturas;
```

3.4. Orde e correspondencia dos valores

Cando se indican as columnas, os valores de `VALUES` asígnanse seguindo exactamente esa orde. Non é obrigatorio manter a orde física das columnas da táboa.

Exemplo

```
INSERT INTO alumnos (
    email,
    nome,
    data_alta,
    curso_id
)
VALUES (
    'ana@example.com',
    'Ana García',
    '2026-09-15',
    1
);
```

Aínda que `email` aparece antes que `nome`, os valores asígnanse correctamente porque se respectou a orde indicada na lista de columnas.

Evitar sempre que sexa posible: inserir valores sen indicar as columnas. Se se omite a lista de columnas, é necesario proporcionar un valor para todas as columnas da táboa e respectar a súa orde de definición. Ademais, un cambio posterior na estrutura pode provocar que a sentenza deixe de funcionar.

4. Inserción especificando columnas, NULL e valores por defecto

4.1. Omitir columnas opcionais

Cando unha columna admite valores por defecto ou `NULL`, pode omitirse da lista de columnas. MariaDB asignará o valor por defecto se existe; noutro caso, asignará `NULL` se a columna o permite.

Exemplo

```
INSERT INTO alumnos (
    nome,
    data_alta,
    curso_id
)
VALUES (
    'Luis López',
    '2026-09-16',
    1
);
```

Neste caso, `email` queda a `NULL` porque admite valores nulos e `activo` recibe o seu valor por defecto, `TRUE`.

4.2. O valor NULL

`NULL` representa a ausencia dun valor coñecido ou aplicable. Non equivale a cero, a unha cadea baleira nin ao texto `'NULL'`.

```
INSERT INTO alumnos (
    nome,
    email,
    data_alta
)
VALUES (
    'Marta Fernández',
    NULL,
    '2026-09-17'
);
```

Para comprobar se unha columna contén valores nulos utilízanse os operadores `IS NULL` e `IS NOT NULL`.

```
-- Alumnos sen correo electrónico
SELECT nome
FROM alumnos
WHERE email IS NULL;

-- Alumnos cun correo electrónico
SELECT nome
FROM alumnos
WHERE email IS NOT NULL;
```

> **Erro frecuente:** non se debe escribir `email = NULL` para buscar valores nulos. Débese utilizar `IS NULL`. As comparacións habituais non funcionan cos valores NULL do mesmo xeito que cos valores normais.

4.3. Valores por defecto

Unha columna pode ter un valor predeterminado definido mediante a cláusula `DEFAULT`. Ese valor utilízase cando se omite a columna na inserción ou cando se indica expresamente a palabra `DEFAULT`.

```
-- Utiliza o valor por defecto de activo
INSERT INTO alumnos (nome, data_alta)
VALUES ('Pedro Rodríguez', '2026-09-18');

-- Indica expresamente que se utilice o valor por defecto
INSERT INTO alumnos (nome, data_alta, activo)
VALUES ('Laura Martínez', '2026-09-19', DEFAULT);
```

4.4. Campos AUTO_INCREMENT

Unha columna declarada como `AUTO_INCREMENT` xera automaticamente un identificador cando se insire un rexistro sen proporcionar un valor para ela. É habitual utilizar este tipo de columnas como claves primarias.

```
INSERT INTO cursos (nome)
VALUES ('ASIR1');

SELECT LAST_INSERT_ID();
```

A función `LAST_INSERT_ID()` devolve o último identificador xerado na conexión actual. Resulta útil cando se necesita coñecer o identificador asignado a un rexistro que acaba de inserirse.

Tamén se pode indicar `NULL` ou `0` para solicitar a xeración automática nas configuracións habituais de MariaDB, aínda que a forma máis clara e recomendable é omitir a columna.

5. Inserción de múltiples rexistros

5.1. Varios rexistros cunha única sentenza

Pódense inserir varias filas nunha única operación separando cada conxunto de valores mediante comas. Esta forma reduce o número de sentenzas que é necesario executar.

Exemplo

```
INSERT INTO cursos (nome)
VALUES
    ('DAW2'),
    ('DAM2'),
    ('ASIR2');
```

5.2. Inserción de varios alumnos

```
INSERT INTO alumnos (
    nome,
    email,
    data_alta,
    curso_id
)
VALUES
    ('Carlos Sánchez', 'carlos@example.com', '2026-09-20', 1),
    ('Elena González', 'elena@example.com', '2026-09-20', 1),
    ('Javier Pérez', 'javier@example.com', '2026-09-21', 2);
```

Cada fila debe conter o mesmo número de valores e manter a correspondencia coas columnas indicadas. Ademais, todos os rexistros deben cumprir as restricións da táboa.

5.3. Inserir datos a partir dunha consulta

MariaDB permite inserir nunha táboa o resultado dunha consulta mediante `INSERT INTO ... SELECT`. Esta operación estudarase con máis detalle no bloque dedicado ás subconsultas e ás ordes de edición.

```
-- Exemplo de estrutura:
-- require que exista unha táboa de destino compatible.

INSERT INTO alumnos_copia (id, nome)
SELECT id, nome
FROM alumnos;
```

> **Idea clave:** en `INSERT ... SELECT` non se utiliza a cláusula `VALUES` . Os rexistros que se insiren proceden das filas devoltas pola consulta.

6. Actualización de rexistros: UPDATE

6.1. Sintaxe básica

A sentenza `UPDATE` permite modificar os valores dunha ou varias columnas dos rexistros que cumpren unha determinada condición.

```
UPDATE nome_taboa
SET columna1 = valor1,
    columna2 = valor2
WHERE condicion;
```

A cláusula `SET` indica que columnas se modifican e que valores reciben. A cláusula `WHERE` determina que rexistros se ven afectados.

6.2. Actualizar un rexistro

Exemplo

```
UPDATE alumnos
SET email = 'ana.garcia@example.com'
WHERE id = 1;
```

A sentenza cambia o correo electrónico da alumna cuxo identificador é 1. Os demais campos permanecen sen modificacións.

6.3. Actualizar varios campos

Unha mesma sentenza pode modificar varias columnas dun rexistro.

```
UPDATE alumnos
SET email = 'luis.lopez@example.com',
    activo = TRUE
WHERE id = 2;
```

6.4. Expresións na cláusula SET

O novo valor dunha columna tamén pode calcularse a partir do seu valor actual.

```
UPDATE matriculas
SET nota = nota + 0.5
WHERE nota IS NOT NULL;
```

A operación anterior incrementa en medio punto todas as cualificacións que non sexan nulas. Cómpre comprobar que o cálculo respecta as restricións definidas para a columna.

7. Condicións nas actualizacións

7.1. Uso de WHERE

A cláusula `WHERE` permite seleccionar os rexistros que se van modificar. Pode utilizar operadores de comparación, operadores lóxicos e outras condicións.

Operador	Significado	Exemplo
`=`	Igual	`curso_id = 1`
`<>` ou `!=`	Distinto	`activo <> TRUE`
`>` , `<`	Maior ou menor	`nota > 5`
`>=` , `<=`	Maior ou igual, menor ou igual	`nota >= 5`
`BETWEEN`	Dentro dun intervalo, incluídos os extremos	`nota BETWEEN 5 AND 10`
`IN`	Pertence a unha lista de valores	`curso_id IN (1, 2)`
`LIKE`	Coincide cun patrón	`nome LIKE 'Ana%'`
`IS NULL`	O valor é nulo	`email IS NULL`

7.2. Operadores lóxicos AND, OR e NOT

Os operadores lóxicos permiten combinar varias condicións.

- `AND` : todas as condicións deben cumprirse.
- `OR` : abonda con que se cumpra algunha das condicións.
- `NOT` : inverte o resultado dunha condición.

Exemplos

```
-- Alumnos activos do curso 1
UPDATE alumnos
SET email = 'pendente@example.com'
WHERE curso_id = 1 AND activo = TRUE;

-- Alumnos dos cursos 1 ou 2
UPDATE alumnos
SET activo = FALSE
WHERE curso_id = 1 OR curso_id = 2;

-- Alumnos que non teñen correo electrónico
UPDATE alumnos
SET email = 'sen-correo@example.com'
WHERE email IS NULL;
```

7.3. Actualización de varios rexistros

Se a condición coincide con varias filas, MariaDB actualizará todas elas. Polo tanto, non se debe asumir que unha sentenza `UPDATE` modifica sempre un único rexistro.

```
UPDATE alumnos
SET activo = FALSE
WHERE curso_id = 2;
```

Esta sentenza desactiva todos os alumnos asociados ao curso 2.

> **Precaución con UPDATE:** se se omite a cláusula `WHERE`, actualizaranse todas as filas da táboa. Antes de executar unha actualización importante, é recomendable comprobar que rexistros selecciona a condición mediante unha consulta `SELECT` equivalente.

Comprobación previa

```
-- Primeiro compróbanse as filas afectadas
SELECT id, nome, activo
FROM alumnos
WHERE curso_id = 2;

-- Despois execútase a modificación
UPDATE alumnos
SET activo = FALSE
WHERE curso_id = 2;
```

8. Eliminación de rexistros: DELETE

8.1. Sintaxe básica

A sentenza `DELETE` elimina filas dunha táboa. A cláusula `WHERE` determina que rexistros se deben borrar.

```
DELETE FROM nome_taboa
WHERE condicion;
```

8.2. Eliminar un rexistro

Exemplo

```
DELETE FROM alumnos
WHERE id = 5;
```

Elimina o alumno co identificador 5, sempre que non existan rexistros relacionados que impidan o borrado polas restricións de clave foránea.

8.3. Eliminar varios rexistros

Do mesmo xeito que ocorre con `UPDATE`, a condición pode coincidir con varias filas.

```
DELETE FROM alumnos
WHERE activo = FALSE;
```

Esta sentenza elimina todos os alumnos inactivos, sempre que non existan referencias que impidan a operación.

8.4. Eliminar todos os rexistros dunha táboa

Se se executa `DELETE` sen cláusula `WHERE`, elimínanse todas as filas da táboa, pero a súa estrutura permanece.

```
DELETE FROM matriculas;
```

Nas táboas InnoDB, esta operación pode formar parte dunha transacción e os seus cambios poden desfacerse mediante `ROLLBACK` antes de confirmar a transacción.

> **Precaución con DELETE:** unha sentenza sen `WHERE` elimina todos os rexistros. Antes de executala, comproba a táboa e a condición. Nas operacións importantes, utiliza transaccións cando o motor e o contexto o permitan.

9. Diferenzas entre DELETE, TRUNCATE e DROP

Aínda que as tres sentenzas poden provocar a desaparición de información, non realizan a mesma operación.

Sentenza	Que elimina?	Mantén a táboa?	Características
`DELETE`	Unha ou varias filas, segundo a condición.	Si.	Admite `WHERE`. En InnoDB pode desfacerse mediante `ROLLBACK` se aínda non se confirmou a transacción.
`TRUNCATE TABLE`	Todos os rexistros da táboa.	Si.	Non admite `WHERE`. Reinicia normalmente o contador `AUTO_INCREMENT`. É unha operación DDL e provoca unha confirmación implícita.
`DROP TABLE`	A táboa completa, incluídos os datos e a definición.	Non.	Elimina a estrutura. Para volver utilizar a táboa será necesario creala de novo.

Exemplos

```
-- Elimina os alumnos inactivos
DELETE FROM alumnos
WHERE activo = FALSE;

-- Elimina todas as filas de matriculas
TRUNCATE TABLE matriculas;

-- Elimina a táboa completa
DROP TABLE matriculas;
```

Restricións de claves foráneas: MariaDB pode impedir un `TRUNCATE TABLE` se outra táboa contén unha clave foránea que referencia a táboa que se quere baleirar. Nese caso, haberá que estudar as dependencias e utilizar unha alternativa adecuada, como borrar as filas na orde correcta.

10. Integridade referencial nas operacións DML

10.1. Que é a integridade referencial?

A integridade referencial garante que as relacións entre táboas manteñan a súa coherencia. Establécese mediante claves foráneas (`FOREIGN KEY`).

Unha clave foránea dunha táboa filla referencia unha clave primaria ou unha clave única dunha táboa pai. Deste xeito, evítase que unha fila filla faga referencia a un rexistro pai que non existe.

10.2. Exemplo de relación entre táboas

Na base de datos de exemplo, `cursos` é a táboa pai e `alumnos` é a táboa filla. A columna `alumnos.curso_id` referencia a `cursos.id`.

```
CREATE TABLE cursos (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nome VARCHAR(20) NOT NULL
) ENGINE=InnoDB;

CREATE TABLE alumnos (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nome VARCHAR(80) NOT NULL,
    curso_id INT,

    CONSTRAINT fk_alumnos_cursos
        FOREIGN KEY (curso_id)
        REFERENCES cursos(id)
) ENGINE=InnoDB;
```

10.3. Restricións ao inserir

Se se insire un alumno cun valor de `curso_id` que non existe na táboa `cursos`, a operación será rexeitada.

Inserción correcta

```
INSERT INTO cursos (nome)
VALUES ('DAW1');

INSERT INTO alumnos (nome, curso_id)
VALUES ('Ana García', 1);
```

Inserción incorrecta

```
INSERT INTO alumnos (nome, curso_id)
VALUES ('Luis López', 999);
```

Se non existe un curso co identificador 999, MariaDB xera un erro de clave foránea. Non se pode crear unha referencia a un rexistro inexistente.

10.4. Restricións ao actualizar

A integridade referencial tamén se comproba cando se modifica o valor dunha clave foránea. O novo valor debe corresponder a unha fila existente na táboa pai, agás que a columna admita `NULL`.

```
-- Correcto se existe o curso 2
UPDATE alumnos
SET curso_id = 2
WHERE id = 1;

-- Incorrecto se non existe o curso 999
UPDATE alumnos
SET curso_id = 999
WHERE id = 1;
```

10.5. Restricións ao borrar

Se unha fila da táboa pai está sendo referenciada por filas dunha táboa filla, o servidor debe aplicar a política de integridade referencial definida na clave foránea. Se non se estableceu unha acción que permita o borrado, a operación normalmente será rexeitada.

```
-- Pode fallar se existen alumnos asociados ao curso 1
DELETE FROM cursos
WHERE id = 1;
```

11. Políticas ON DELETE e ON UPDATE

11.1. Accións referenciais

Ao crear unha clave foránea pódense definir as accións que se aplicarán cando se modifique ou elimine o valor referenciado na táboa pai.

Acción	Comportamento	Uso habitual
`RESTRICT`	Impide modificar ou borrar a fila pai se existen filas fillas relacionadas.	Evitar a eliminación de información que aínda ten dependencias.
`NO ACTION`	En InnoDB de MariaDB compórtase como `RESTRICT`.	Indicar que non se permite a operación se rompe a relación.
`CASCADE`	Propaga a modificación ou eliminación ás filas relacionadas.	Manter automaticamente a coherencia cando a fila filla non debe existir sen a pai.
`SET NULL`	Establece a `NULL` a clave foránea das filas fillas.	Conservar as filas fillas aínda que deixen de estar asociadas á fila pai.

11.2. ON DELETE

A cláusula `ON DELETE` determina que sucede coas filas fillas cando se elimina a fila pai.

Exemplo con ON DELETE CASCADE

```
CREATE TABLE cursos (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nome VARCHAR(20) NOT NULL
) ENGINE=InnoDB;

CREATE TABLE alumnos (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nome VARCHAR(80) NOT NULL,
    curso_id INT NOT NULL,

    CONSTRAINT fk_alumnos_cursos
        FOREIGN KEY (curso_id)
        REFERENCES cursos(id)
        ON DELETE CASCADE
        ON UPDATE CASCADE
) ENGINE=InnoDB;
```

Se se elimina un curso, MariaDB elimina tamén os alumnos asociados a ese curso. Esta propagación realízase automaticamente.

```
DELETE FROM cursos
WHERE id = 1;
```

11.3. ON DELETE SET NULL

Se se utiliza `SET NULL`, as filas fillas consérvanse, pero a súa clave foránea pasa a ser nula. Polo tanto, a columna da táboa filla debe admitir valores `NULL`.

```
CREATE TABLE alumnos (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nome VARCHAR(80) NOT NULL,
    curso_id INT NULL,

    CONSTRAINT fk_alumnos_cursos
        FOREIGN KEY (curso_id)
        REFERENCES cursos(id)
        ON DELETE SET NULL
) ENGINE=InnoDB;
```

11.4. ON UPDATE

A cláusula `ON UPDATE` define que sucede cando se modifica o valor da clave referenciada na táboa pai.

```
FOREIGN KEY (curso_id)
REFERENCES cursos(id)
ON UPDATE CASCADE;
```

Con `ON UPDATE CASCADE`, se cambia o identificador dun curso, MariaDB actualiza automaticamente as claves foráneas dos alumnos relacionados.

11.5. Escoller unha política adecuada

A política depende do significado dos datos e das regras do sistema:

- **RESTRICT:** apropiado cando non se debe eliminar un elemento que ten información dependente.
- **CASCADE:** apropiado cando os rexistros fillos carecen de sentido sen o rexistro pai, por exemplo, as liñas dun pedido que se elimina.
- **SET NULL:** apropiado cando se desexa conservar o rexistro fillo, pero desvinculalo do pai.

> **Atención:** `CASCADE` pode provocar a eliminación de moitas filas mediante unha única sentenza. Antes de definir esta política, hai que valorar se esa eliminación automática é realmente a regra de negocio desexada.

12. Resolución de erros de integridade referencial

12.1. Identificar a táboa pai e a táboa filla

Cando unha operación DML produce un erro de clave foránea, o primeiro paso consiste en identificar que táboa contén a referencia e que táboa contén o rexistro referenciado.

Pódese consultar a definición dunha táboa mediante:

```
SHOW CREATE TABLE alumnos;
```

Tamén se pode consultar a información das restricións no catálogo `INFORMATION_SCHEMA`.

```
SELECT
    TABLE_NAME,
    CONSTRAINT_NAME,
    COLUMN_NAME,
    REFERENCED_TABLE_NAME,
    REFERENCED_COLUMN_NAME
FROM INFORMATION_SCHEMA.KEY_COLUMN_USAGE
WHERE TABLE_SCHEMA = DATABASE()
  AND REFERENCED_TABLE_NAME IS NOT NULL;
```

12.2. Orde correcta de inserción

Cando se insiren datos relacionados, primeiro débense crear os rexistros das táboas pai e despois os das táboas fillas. Deste xeito, as claves foráneas poden apuntar a rexistros que xa existen.

Exemplo de inserción ordenada

```
-- 1. Primeiro insírese o curso (táboa pai)
INSERT INTO cursos (nome)
VALUES ('DAW1');

-- 2. Despois insírese o alumno (táboa filla)
INSERT INTO alumnos (nome, data_alta, curso_id)
VALUES ('Ana García', '2026-09-15', 1);

-- 3. Finalmente rexístrase a matrícula
INSERT INTO matriculas (
    alumno_id,
    asignatura_id,
    data,
    nota
)
VALUES (1, 1, '2026-09-16', NULL);
```

A última operación require que existan previamente tanto o alumno co identificador 1 como a asignatura co identificador 1.

12.3. Orde correcta de eliminación

Cando se eliminan rexistros relacionados e se utiliza unha política restritiva, a orde habitual é a inversa á de inserción: primeiro elimínanse as filas das táboas fillas e despois as das táboas pai.

```
-- 1. Eliminar primeiro as matrículas do alumno
DELETE FROM matriculas
WHERE alumno_id = 1;

-- 2. Despois eliminar o alumno
DELETE FROM alumnos
WHERE id = 1;

-- 3. Finalmente, se xa non ten alumnos asociados,
-- eliminar o curso
DELETE FROM cursos
WHERE id = 1;
```

12.4. Diagnóstico de erros frecuentes

Situación	Posible causa	Que comprobar?
Erro ao inserir unha fila filla.	O valor da clave foránea non existe na táboa pai.	Que o rexistro pai exista e que o identificador utilizado sexa correcto.
Erro ao actualizar unha clave foránea.	O novo valor non existe na táboa referenciada.	Que o valor de destino exista ou que a columna poida admitir `NULL`.
Erro ao borrar unha fila pai.	Hai filas fillas que a referencian e a política impide o borrado.	Localizar as filas relacionadas e revisar `ON DELETE`.
Erro ao inserir un valor duplicado.	Incúmprese unha clave primaria ou unha restrición `UNIQUE`.	Comprobar se xa existe o identificador ou o valor único.
Erro por valor nulo.	Inténtase inserir `NULL` nunha columna `NOT NULL`.	Revisar se se debe proporcionar un valor ou definir un valor por defecto.

13. Boas prácticas ao modificar datos

1. Seleccionar e comprobar a base de datos antes de executar operacións.
2. Indicar explicitamente as columnas nas sentenzas `INSERT`.
3. Utilizar condicións `WHERE` precisas en `UPDATE` e `DELETE`.
4. Antes de modificar ou borrar, executar un `SELECT` coa mesma condición para comprobar as filas afectadas.
5. Respectar as restricións de claves primarias, claves foráneas, valores únicos e columnas obrigatorias.

6. Inserir primeiro os rexistros pai e despois os rexistros fillos.
7. Eliminar primeiro os rexistros fillos cando as regras de integridade así o esixan.
8. Escoller as accións `ON DELETE` e `ON UPDATE` segundo as regras reais da aplicación.
9. Evitar utilizar `TRUNCATE` ou `DROP` sen comprobar antes as súas consecuencias.
10. Utilizar transaccións nas operacións que requiran poder confirmar ou desfacer os cambios. Este mecanismo estudarase nun bloque posterior.

14. Resumo das sentenzas

Operación	Sentenza	Exemplo
Inserir un rexistro	`INSERT INTO`	`INSERT INTO cursos (nome) VALUES ('DAW1');`
Inserir varios rexistros	`INSERT INTO ... VALUES`	`INSERT INTO cursos (nome) VALUES ('DAW1'), ('DAW2');`
Inserir o resultado dunha consulta	`INSERT INTO ... SELECT`	`INSERT INTO destino (nome) SELECT nome FROM orixe;`
Actualizar rexistros	`UPDATE ... SET`	`UPDATE alumnos SET activo = FALSE WHERE id = 1;`
Eliminar rexistros	`DELETE FROM`	`DELETE FROM alumnos WHERE id = 1;`
Baleirar unha táboa	`TRUNCATE TABLE`	`TRUNCATE TABLE matriculas;`
Eliminar unha táboa	`DROP TABLE`	`DROP TABLE matriculas;`

15. Guión de traballo e comprobación

Para practicar os contidos deste bloque, recoméndase seguir esta orde de traballo en MariaDB:

1. Crear ou seleccionar a base de datos de exemplo.
2. Consultar a estrutura das táboas con `DESCRIBE` e `SHOW CREATE TABLE`.
3. Inserir rexistros nas táboas pai.
4. Inserir rexistros nas táboas fillas utilizando identificadores existentes.
5. Consultar os rexistros para comprobar as insercións.

6. Actualizar un ou varios rexistros utilizando condicións específicas.
7. Comprobar previamente con `SELECT` que filas se verán afectadas por cada actualización ou borrado.
8. Probar unha inserción cunha clave foránea inexistente e analizar o erro obtido.
9. Probar o borrado dun rexistro pai que teña filas fillas relacionadas.
10. Revisar as regras `ON DELETE` e `ON UPDATE` das claves foráneas.
11. Eliminar os rexistros na orde adecuada e comprobar o estado final das táboas.

> **Obxectivo do bloque:** ser capaz de inserir, actualizar e eliminar información mediante SQL, comprendendo como afectan estas operacións ás relacións entre táboas e aplicando as medidas necesarias para conservar a integridade e a consistencia dos datos.